

四川大學

本科生课程作业



题 目 软件过程与管理第一次作业

课 程 名 软件过程与管理

任课教师 田攀良

学 院 软件学院

专 业 软件工程

学生姓名 唐嘉

学 号 2023141461011

年 级 2023 级

目录

一、对软件过程与相关标准的理解.....	3
二、项目背景.....	4
三、软件过程总体定义.....	5
四、项目启动与计划过程.....	5
五、需求获取与分析过程.....	6
六、迭代计划过程.....	7
七、设计过程.....	8
八、编码实现过程.....	8
九、测试与质量保证过程.....	9
十、配置管理过程.....	10
十一、评审与验收过程.....	10
十二、部署交付与维护过程.....	10
十三、软件过程定义的来源.....	11
十四、软件过程剪裁的想法.....	12
十五、过程定义后的特点与价值.....	13

一、对软件过程与相关标准的理解

通过阅读课程资料和附件，我对软件过程的理解更加系统。软件过程并不只是“先写需求、再写代码、最后测试”这样简单的步骤安排，而是围绕软件产品从提出想法到交付使用，再到维护改进的完整生命周期所建立的一套活动、角色、产物和管理机制。课程作业要求在阅读 ISO/IEC12207、15504、33001 等相关材料的基础上，定义一个自己做过项目的软件过程，并说明过程来源和剪裁思路，这实际上要求我们把标准中的过程思想转化为具体项目中的可执行做法。

从 ISO/IEC12207 的角度看，软件生命周期过程覆盖了需求、设计、实现、集成、测试、交付、维护等基本工程活动，同时也包括项目管理、配置管理、质量保证、评审、问题解决等支持性活动。这说明软件开发不是单一的编码活动，而是由多个相互关联的过程共同组成。一个项目如果只关注代码实现，而忽视需求确认、版本控制、测试验证和交付管理，就容易出现返工、质量不稳定和交付不可控的问题。

ISO/IEC15504 更加强调过程能力评估。也就是说，一个团队不仅要定义过程，还要关注过程是否真正执行，执行是否稳定，产物是否可检查，结果是否能够持续改进。过程能力的思想对课程项目也有启发意义。即使项目规模不大，也应当保留必要的过程证据，例如需求记录、设计文档、代码提交记录、测试记录和缺陷修复记录。只有这样，项目结束后才能判断哪些过程有效，哪些过程需要改进。

ISO/IEC33001 在过程评估和持续改进方面进一步扩展了相关思想。它提醒我们，软件过程不应被看作一次性规定，而应当根据组织目标、项目特点、风险水平和团队能力不断调整。对于学生课程项目而言，完全照搬大型组织的过程模型并不现实，但可以吸收其中关于生命周期、过程能力、过程评估和持续改进的核心思想，形成适合小型团队的轻量级过程。

在阅读附件材料时，我也注意到当前软件工程实践正在受到敏捷方法、AI 工具和自动化技术的影响。例如，关于 AI 驱动敏捷开发教育的论文指出，学生只有在真实项目中经历团队协作、需求变化、持续反馈和质量责任，才能真正理解敏捷软件开发，而不是停留在课堂概念层面。该文还提到，项目团队通常通过多个 Sprint 推进需求澄清、架构设计、编码、测试和文档工作，并强调人类仍然需要对质量、验收标准和责任承担最终判断。这与软件过程管理课程中的思想一致，即过程既要规范开发活动，又要服务于项目实际目标。

二、项目背景



图 1 校园任务协作与课程项目管理系统用户角色图

我选择定义软件过程的项目是自己曾经参与完成的校园任务协作与课程项目管理系统。该系统面向课程项目小组、教师和助教，主要用于解决课程项目开发过程中任务分配不清、进度不可见、文件分散、沟通成本较高等问题。系统希望让学生小组能够在一个统一平台中创建项目、分配任务、跟踪进度、提交文档，并让教师或助教能够查看项目整体完成情况。

该系统的主要用户包括学生、组长、教师和助教。学生可以查看自己负责的任务、更新任务状态、上传项目文档和参与讨论；组长可以创建项目、分配任务、调整成员职责和跟踪整体进度；教师和助教可以查看各小组项目情况，检查阶段性成果，并对项目过程进行评价。系统的核心功能包括用户登录、角色权限管理、项目创建、任务分配、任务状态跟踪、文件上传、通知提醒、项目进度统计和项目总结展示。

这个项目的规模属于中小型课程项目，团队人数有限，开发周期较短，需求在开发过程中也会根据教师意见和团队讨论不断调整。如果采用严格瀑布模型，前期需要投入较多时间编写完整需求和详细设计文档，但课程项目往往很难在一开始就完全确定所有需求。如果完全采用自由式开发，又容易导致需求不清、接口不一致、测试不足和最终交付混乱。因此，我认为该项目更适合采用一种轻量级迭代式软件过程，即以 ISO/IEC12207 的生命周期过程为基础，以敏捷迭代为主要开发节奏，同时结合 ISO/IEC15504 和 ISO/IEC33001 的过程评估与持续改进思想进行适当剪裁。

三、软件过程总体定义



图 2 轻量级迭代式软件过程总体模型

针对上一章提到的校园任务协作与课程项目管理系统，我将软件过程定义为一个轻量级迭代式开发过程。该过程从项目启动开始，经过需求分析、迭代计划、设计、实现、测试、评审、部署和维护总结等阶段完成整个项目。与传统瀑布模型不同的是，这些活动并不是一次性顺序完成，而是在多个迭代周期中不断重复和细化。每一轮迭代都需要选择一部分高优先级需求，完成相应设计、编码、测试和评审，并形成可以运行、可以演示、可以检查的阶段性版本。

在总体流程上，项目首先进入启动与计划阶段，明确项目目标、团队分工、开发周期和主要风险。随后进入需求分析阶段，通过讨论、场景分析和用户故事整理项目需求。然后按照需求优先级制定迭代计划，每轮迭代完成一组功能。进入开发阶段后，团队根据设计结果进行编码实现，并通过单元测试、接口测试和集成测试保证质量。每轮迭代结束后进行评审和反馈，根据反馈调整后续任务。最终，在系统功能基本稳定后完成部署交付，并在项目结束时进行维护总结和过程复盘。

这个过程的核心思想是“标准过程提供框架，敏捷迭代提供节奏，轻量文档提供证据，测试评审保障质量，项目复盘促进改进”。它既不是完全形式化的重型过程，也不是缺乏管理的自由开发，而是在规范性和灵活性之间取得平衡。

四、项目启动与计划过程

项目启动与计划是整个软件过程的起点。在这个阶段，团队首先需要明确为什么要开发这个系统，系统要解决什么问题，项目完成后应当达到什么效果。对于本项目而言，系统目标是为课程项目小组提供统一的协作管理平台，让任务、文档、进度和反馈能够被集中管理，从而提高小组协作效率和教师检查效率。

在启动阶段，团队还需要明确项目范围。由于课程项目开发时间有限，不可能实现完整企业级项目管理平台的所有能力，因此系统范围被限定在课程小组协作场景内。项目不追求复杂的组织级权限体系，也不实现大型项目管理软件中的预算管理、人力资源管理和复杂报表分析，而是重点支持课程项目中最常见的任务分配、进度跟踪、文档管理和阶段评审。

团队分工也是启动阶段的重要内容。由于团队人数较少，不能像企业项目那样设置完整的项目经理、需求工程师、架构师、开发人员、测试人员、配置管理员和质量保证人员。因此，本项目采用角色合并的方式，由一名同学负责整体协调和进度跟踪，部分同学负责前端页面开发，部分同学负责后端接口和数据库设计，另有同学负责测试、文档整理和演示材料准备。虽然角色进行了合并，但每项关键职责仍然要有人负责，这样可以避免出现“过程上有人做，实际上没人管”的情况。

项目启动阶段的主要产物包括项目目标说明、项目范围说明、团队分工表、初步计划、风险清单和协作环境。协作环境包括代码仓库、文档目录、任务看板和团队沟通渠道。这些产物不需要写成冗长正式文档，但必须能够让团队成员清楚知道项目做什么、谁来做、什么时候完成、怎样协作以及出现问题如何处理。

五、需求获取与分析过程

需求获取与分析过程的目标是明确系统真正需要实现的功能和质量要求。对于本项目来说，需求主要来自课程项目管理中的实际痛点，包括小组任务分配不明确、成员完成情况不透明、文档版本混乱、教师难以及时了解项目进度等。因此，需求分析应当围绕真实使用场景展开，而不是简单罗列功能名称。

在需求分析过程中，我首先将系统用户划分为学生、组长、教师和助教，并分别分析他们的使用目标。学生关注的是自己需要完成哪些任务、任务什么时候截止、如何提交成果；组长关注的是如何创建项目、分配任务、查看小组整体进度；教师和助教关注的是如何查看各组项目状态、检查阶段成果并进行反馈。通过这种角色分析，可以避免系统功能只从开发者角度出发，而忽略真实用户需求。

随后，团队将需求表达为用户故事。例如，作为组长，我希望能够创建任务并分配给小组成员，以便明确每个人的工作职责。作为学生，我希望能够查看自己的待办任务和截止时间，以便合理安排工作。作为教师，我希望能够查看每个小组的任务完成情况，以便及时发现进度滞后的项目。用户故事的优点是语言自然，便于团队成员理解，也便于后续转化为开发任务和验收标准。

需求分析不仅包括功能需求，也包括非功能需求。本项目的非功能需求包括系统界面应当简单易用，常用操作响应速度应当较快，用户权限应当清晰，文档上传和下载应当可靠，系统数据不能随意被未授权用户访问。虽然这些非功能需求在课程项目中常常被忽略，但它们直接影响系统是否真正可用。

为了支持后续开发和测试，需求分析阶段还需要形成需求跟踪关系。每个需求应当有编号、描述、优先级、状态和对应测试方式。需求跟踪不需要使用复杂工具，可以通过表格维护。这样做的好处是，在项目后期可以检查每个核心需求是否已经实现，是否经过测试，是否通过验收。附件中关于需求到 UML 类图生成的研究也说明，图形模型能够帮助利益相关者理解、交流和验证系统需求，UML 等模型在需求分析和系统设计中具有重要作用。因此，本项目在需求分析后也绘制了用例图和简单领域模型，用于统一团队对系统结构的理解。

六、迭代计划过程

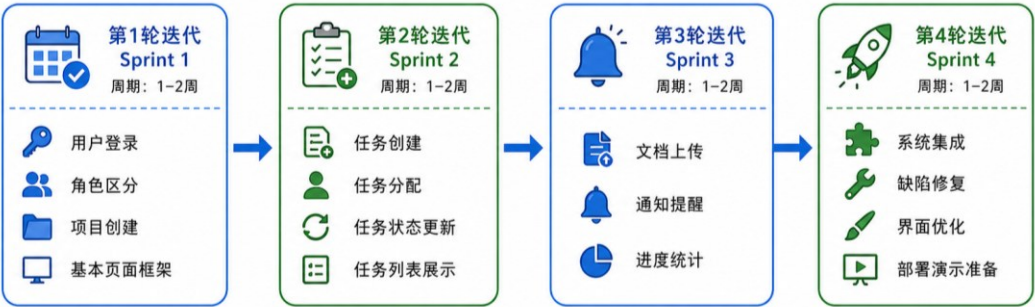


图 3 项目迭代开发计划示意图

由于项目需求会逐步清晰，团队开发能力和时间也有限，因此本项目没有采用一次性完成全部需求的方式，而是将开发过程划分为多个短迭代。每个迭代周期大约为一到两周，每轮迭代开始时选择若干优先级较高的用户故事，并将其拆分为具体开发任务。

第一轮迭代主要完成系统基础能力，例如用户登录、角色区分、项目创建和基本页面框架。第二轮迭代重点完成任务管理，包括任务创建、任务分配、任务状态更新和任务列表展示。第三轮迭代完成文档上传、通知提醒和进度统计等功能。最后一轮迭代主要用于系统集成、缺陷修复、界面优化、部署和最终演示准备。

迭代计划过程的关键是明确每轮迭代的目标和完成标准。所谓完成标准，不只是代码写完，而是功能可以运行，接口可以调用，基本测试通过，相关文档得到更新，并且能够在迭代评审中演示。这样可以防止团队把“写了代码”误认为“完成了功能”。

本项目借鉴了 Scrum 中产品待办列表、SprintBacklog 和迭代评审的思想，但没有完整照搬 Scrum 框架。例如，项目没有设置专职 ScrumMaster，也没有每天举行严格站会，而是采用每两到三天一次的简短同步方式，检查任务进展、接口问题和风险事项。这种剪裁更适合学生团队，因为它保留了敏捷开发的核心节奏，同时减少了形式化会议成本。

七、设计过程

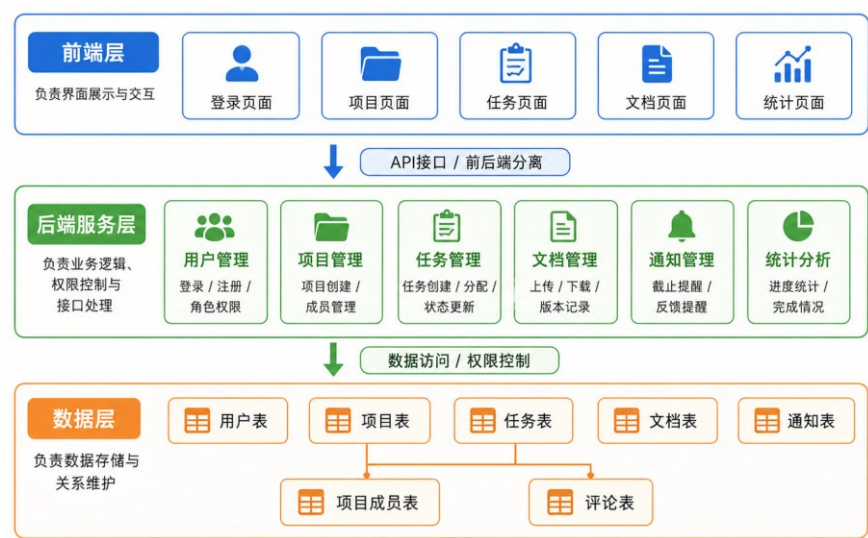


图 4 校园任务协作系统总体架构图

设计过程的目标是将需求转化为可以指导实现的系统结构。本项目采用前后端分离架构。前端负责用户界面展示和交互，后端负责业务逻辑、数据访问和权限控制，数据库负责保存用户、项目、任务、文档和通知等数据。这样的架构清晰、易于分工，也适合课程项目团队并行开发。

在总体设计中，系统被划分为用户管理、项目管理、任务管理、文档管理、通知管理和统计分析几个模块。用户管理模块负责登录、注册、角色识别和权限控制；项目管理模块负责项目创建、小组成员管理和项目基本信息维护；任务管理模块负责任务创建、分配、状态更新和任务查询；文档管理模块负责项目资料上传、下载和版本记录；通知管理模块负责任务截止提醒和教师反馈提醒；统计分析模块负责展示项目进度和成员任务完成情况。

数据库设计是本项目设计过程中的重点。团队需要设计用户表、项目表、项目成员表、任务表、文档表、通知表和评论表等核心数据表，并明确表之间的关系。例如，一个项目可以包含多个成员，一个项目可以包含多个任务，一个任务可以分配给一个或多个成员，一个项目可以拥有多份文档。通过 E-R 图和表结构说明，团队可以提前发现数据关系中的问题，减少后期返工。

接口设计同样重要。由于项目采用前后端分离，如果接口定义不清，前端和后端开发很容易相互等待或反复修改。因此，本项目在编码前先定义主要 API，包括登录接口、项目接口、任务接口、文档接口和统计接口。接口文档需要说明请求地址、请求方式、参数含义、返回格式和错误情况。接口文档不需要过度复杂，但必须保证前后端成员理解一致。

设计过程还包括对关键方案的评审。评审的重点不是追求设计形式，而是检查模块划分是否合理，数据结构是否支持需求，接口是否完整，权限控制是否存在漏洞，后续实现是否可行。对于课程项目来说，设计评审可以通过小组会议完成，并形成简单记录。

八、编码实现过程

编码实现过程是将设计转化为可运行软件的阶段。本项目要求团队成员按照

模块分工进行开发，并遵守统一的代码规范。代码规范包括目录结构、变量命名、接口命名、注释要求、异常处理和提交信息格式等。统一规范的目的不是增加形式负担，而是降低多人协作时的理解成本。

项目使用 **Git** 进行版本管理。主分支保存稳定版本，开发分支用于日常集成，个人功能分支用于具体功能开发。开发人员完成某项任务后，需要提交代码并说明对应任务或需求。重要功能合并到开发分支前，需要至少经过一名其他成员检查。这样可以在早期发现明显问题，减少后期集成风险。

在编码过程中，团队还需要保持代码和文档的一致性。如果接口发生变化，应及时更新接口文档；如果数据库表结构调整，应同步更新数据库设计说明；如果需求实现方式发生较大改变，应在需求记录中说明原因。这样可以保证项目产物之间具有一定的可追踪性。

编码实现不是孤立的过程，它与测试、配置管理和评审密切相关。每次代码提交都应尽量保持小步提交，避免一次提交过多内容导致问题难以定位。对于关键功能，例如权限校验、任务状态变更和文件上传，应当特别注意异常处理和边界情况。

九、测试与质量保证过程

测试与质量保证过程的目标是发现缺陷、验证功能并保证系统满足需求。课程项目中常见的问题是前期只关注实现功能，后期才匆忙测试，最终导致大量缺陷集中暴露。为了避免这种情况，本项目将测试活动嵌入每个迭代，而不是放到项目最后。

测试首先从单元测试开始。后端核心业务逻辑，例如用户权限判断、任务状态更新、项目成员校验和文件记录保存，应当编写基本单元测试。虽然课程项目不一定要求测试覆盖率很高，但关键逻辑必须经过验证。其次是接口测试，前后端联调前，后端应先通过接口测试工具验证主要接口是否能够正常返回数据。再次是集成测试，即检查多个模块协同工作时是否符合预期，例如创建项目后是否能够添加成员，添加成员后是否能够分配任务，任务状态更新后统计结果是否同步变化。

验收测试则根据用户故事进行。例如，对于“组长可以创建任务并分配给成员”这一用户故事，验收测试应检查组长是否能够进入任务创建页面，是否能够填写任务标题、描述、截止时间和负责人，任务创建后相关成员是否能够看到该任务，任务状态变化是否能够被组长查看。这样的测试方式能够保证测试与需求直接关联。

质量保证过程还包括缺陷管理。发现缺陷后，需要记录缺陷现象、复现步骤、严重程度、责任人、修复状态和验证结果。缺陷记录可以采用简单表格或项目管理工具完成。重要的是缺陷不能只停留在口头沟通中，否则容易遗漏或重复出现。

附件中关于 **AI** 代理生成微服务的研究也提示我们，自动生成或快速实现的软件并不意味着可以完全信任，其功能正确性、接口契约和架构一致性仍然需要测试和人工监督。该研究指出，**AI** 代理虽然可以生成可维护代码，但正确性并不稳定，仍然需要人工确保 **API** 和架构一致性。这一点同样适用于本项目：无论代码由人工完成还是借助工具生成，都必须经过测试、评审和验收。

十、配置管理过程

配置管理过程的目标是保证项目中的代码、文档、版本和变更处于可控状态。本项目虽然规模不大，但仍然需要配置管理，因为多人协作开发很容易出现代码冲突、文档版本混乱和最终交付版本不可复现的问题。

在本项目中，配置管理主要通过 Git 仓库和统一文档目录实现。代码、数据库脚本、接口文档、需求文档、设计文档、测试记录和部署说明都应纳入项目目录管理。重要版本需要打标签，例如第一次可运行版本、阶段演示版本和最终提交版本。这样即使后续修改出现问题，也可以回到稳定版本。

变更管理也是配置管理的一部分。项目开发过程中，需求变化不可避免。例如，教师可能要求增加项目进度统计，团队可能发现某些功能实现成本过高，需要调整范围。对于这些变化，团队应记录变更原因、影响范围和处理结果。对于小型课程项目，变更记录可以很简单，但不能完全没有。否则后期很难解释为什么某个功能被增加、删除或调整。

配置管理的剪裁思路是保留核心控制点，删除复杂审批流程。也就是说，本项目不建立正式配置控制委员会，不使用复杂配置管理系统，但通过仓库权限、分支策略、提交记录、版本标签和变更表实现基本控制。

十一、评审与验收过程

评审与验收过程用于判断阶段成果是否满足需求，并为后续迭代提供反馈。每轮迭代结束后，团队需要进行一次功能演示，展示本轮完成的功能，并根据事先定义的完成标准检查功能是否真正完成。评审内容包括功能是否可用，界面是否符合使用习惯，接口是否稳定，测试是否通过，文档是否同步更新。

评审过程中发现的问题不应简单地口头讨论后结束，而应记录到待办列表中，并安排到后续迭代中处理。例如，如果教师在演示中指出项目进度统计不够直观，团队可以将其记录为改进需求；如果小组成员发现任务状态更新后通知没有同步发送，则应记录为缺陷并安排修复。

最终验收阶段需要检查系统整体功能是否满足项目目标。验收不仅检查功能数量，也检查系统是否稳定、是否能够部署运行、是否有必要文档、是否能够完成典型使用场景。对于本项目而言，最终验收至少应覆盖用户登录、项目创建、成员管理、任务分配、任务状态更新、文档上传、进度查看和教师反馈等核心场景。

评审与验收过程体现的是软件确认思想，即确认系统是否满足用户真实需要，而不是仅仅确认程序能够运行。一个系统即使技术上可以运行，如果不能解决课程项目协作中的实际问题，也不能算真正成功。

十二、部署交付与维护过程

部署交付过程是将系统从开发状态转化为可使用状态。本项目的部署包括后端服务部署、前端页面部署、数据库初始化、配置文件设置和运行环境检查。部署完成后，需要进行冒烟测试，确认系统可以登录、主要页面可以访问、核心接口可以调用、数据库连接正常、文件上传和下载可用。

交付内容不仅包括可运行系统，也包括必要文档。最终交付材料应包括项目

说明、需求文档、设计文档、接口文档、测试记录、部署说明、用户使用说明和项目总结。对于课程项目来说，这些文档也是评价项目过程的重要依据。它们能够反映团队是否按照过程开展工作，而不是只在最后提交代码。

维护过程在课程项目中通常较短，但仍然有必要保留。系统交付后，如果教师或同学在试用过程中发现问题，团队需要记录问题并进行修复。维护结束后，团队应进行项目复盘，分析项目过程中哪些做法有效，哪些做法存在不足。例如，需求记录是否足够清晰，接口文档是否及时更新，测试是否开始得太晚，代码合并是否出现冲突，任务分配是否均衡。复盘结果可以作为下一次项目过程改进的依据。

十三、软件过程定义的来源

本项目软件过程的第一个来源是 ISO/IEC12207 的软件生命周期过程思想。项目中定义的需求、设计、编码、测试、交付和维护过程，均对应生命周期中的基本工程活动。同时，项目管理、配置管理、质量保证、评审和问题处理则体现了标准中的支持性过程和管理性过程。通过参考这一标准，项目过程具有了比较完整的结构，不会只关注编码而忽视其他关键活动。

第二个来源是 ISO/IEC15504 的过程能力思想。该标准启发我在定义过程时不仅写出活动名称，还要明确每个过程的输入、输出和执行证据。例如，需求过程应有需求列表和需求跟踪记录，设计过程应有架构图和接口文档，测试过程应有测试用例和缺陷记录，配置管理过程应有版本记录和变更记录。这些产物使过程是否执行变得可以检查。

第三个来源是 ISO/IEC33001 的持续改进思想。项目过程不应固定不变，而应根据执行情况调整。例如，如果某一轮迭代中发现接口变更频繁，就需要在下一轮加强接口评审；如果发现测试总是滞后，就需要提前设计验收标准；如果发现文档更新不及时，就需要指定文档负责人同步维护。这种根据反馈改进过程的做法，是过程管理真正有价值的地方。

第四个来源是敏捷开发思想。课程项目时间有限、需求变化较快，适合采用短周期迭代和持续反馈。附件中的敏捷项目教学案例也表明，真实项目中的敏捷能力需要通过持续迭代、团队协作、需求澄清、测试和评审来形成，而不只是学习敏捷术语。因此，本项目吸收了用户故事、迭代计划、任务看板、阶段演示和回顾总结等敏捷实践。

第五个来源是本人参与课程项目的实际经验。在实际项目中，如果过程过于复杂，团队难以坚持执行；如果过程过于随意，又会导致需求混乱、接口不一致、测试不足和最终交付困难。因此，本项目过程定义采用“保留必要活动、简化复杂形式、强化关键质量点”的原则。

十四、软件过程剪裁的想法

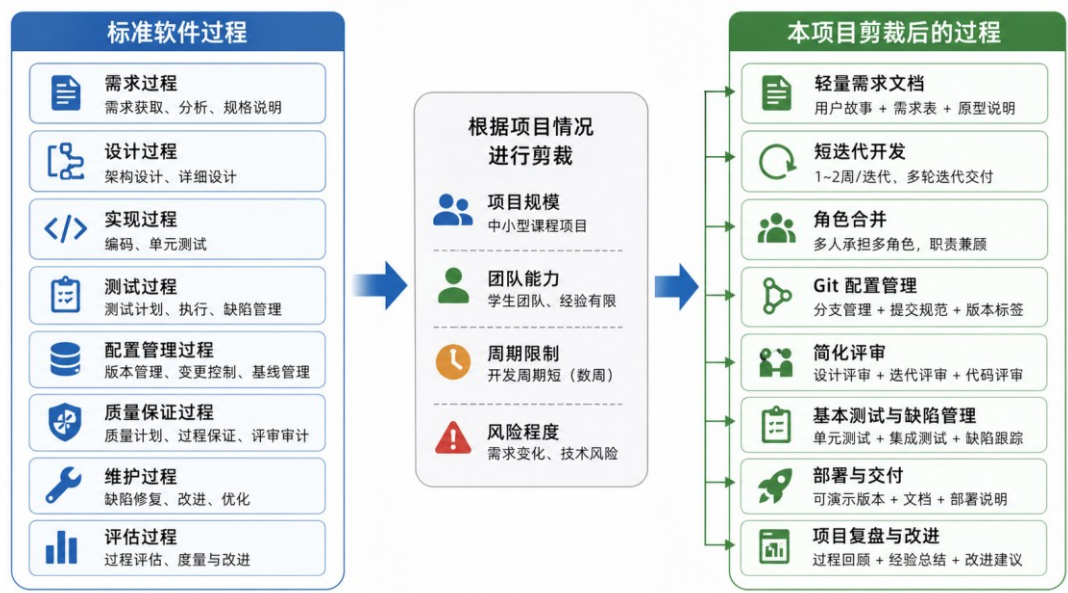


图 5 软件过程剪裁思路示意图

软件过程剪裁是本次作业的重点之一。标准过程通常面向不同规模、不同组织和不同类型的软件项目，内容较为完整。如果不加选择地照搬到课程项目中，会造成过程负担过重，团队可能把大量时间花在形式化文档和流程上，而不是解决项目真正的问题。因此，本项目在剪裁时遵循适用性原则，即保留对项目成功有直接帮助的活动，简化对小团队成本过高的活动，强化对质量和协作影响较大的活动。

首先，项目保留了生命周期主线，但简化了文档形式。需求、设计、编码、测试、交付和维护这些活动不能省略，因为它们构成软件开发的基本逻辑。但是，需求规格说明不采用大型企业项目中的完整模板，而采用用户故事、需求表和原型说明相结合的方式。设计文档也不追求厚重，而以架构图、数据库表结构和接口说明为主。这样既保证必要信息被记录，又不会让团队被文档负担拖慢。

其次，项目强化了需求跟踪。课程项目中经常出现需求一开始说不清、开发中不断变化的情况。如果不记录需求变化，后期就很难判断哪些功能应该实现，哪些功能已经被取消，哪些功能属于新增范围。因此，本项目要求每个核心需求都能关联到开发任务、代码实现、测试用例和验收结果。虽然这种跟踪可以用简单表格完成，但它对控制范围和支撑验收非常重要。

再次，项目采用轻量级迭代方式替代完整瀑布流程。瀑布模型强调阶段顺序和前期完整定义，但本项目需求具有一定不确定性，团队也需要通过原型和演示逐步明确目标。因此，我将开发过程剪裁为多个短迭代，每轮迭代完成部分可运行功能。与此同时，项目并没有完全放弃计划和文档，而是在每轮迭代中保留计划、设计、实现、测试和评审，从而避免敏捷开发被误解为“想到哪里做到哪里”。

在角色设置方面，本项目对标准角色进行了合并。标准过程中可能区分项目经理、需求工程师、架构师、测试人员、配置管理员和质量保证人员，但课程项目团队人数有限，不可能每个角色都设置专人。因此，项目负责人同时承担进度

协调和配置管理职责，开发人员同时承担单元测试职责，文档负责人同时记录需求和整理交付材料，测试负责人组织交叉测试。这样的角色剪裁符合小团队实际，但仍然保证关键职责有人承担。

在质量保证方面，本项目没有设置独立质量保证部门，也没有采用复杂审计机制，但保留了测试、评审和缺陷管理。这样剪裁的原因是，课程项目规模较小，不需要企业级质量体系，但基本质量控制不能省略。特别是本项目涉及用户权限、任务状态、文件上传和进度统计等功能，如果缺乏测试和评审，最终系统可能无法稳定演示和交付。

配置管理也进行了轻量化剪裁。本项目不采用复杂配置管理平台，也不建立正式变更委员会，而是通过 Git 分支、提交规范、版本标签、文档目录和变更记录实现基本管理。这种方式简单可行，适合学生团队执行，同时能够解决版本混乱和交付不可复现的问题。

过程评估方面，本项目没有采用正式的能力等级评定，而是采用项目复盘方式进行简化评估。项目结束后，团队检查各阶段产物是否完整，需求是否得到实现，测试是否覆盖核心场景，缺陷是否修复，迭代计划是否按时完成。通过这些检查，可以判断本次过程执行效果，并形成后续改进建议。这种方式虽然不如正式过程评估严谨，但更适合课程项目。

十五、过程定义后的特点与价值

经过上述定义和剪裁后，本项目软件过程具有较强的适用性。它保留了软件生命周期过程的完整主线，使项目不会只停留在编码层面；它采用迭代开发方式，使团队能够适应需求变化并持续获得反馈；它保留了必要文档和过程证据，使项目执行情况可以被检查；它强化了测试、评审和配置管理，使软件质量和交付稳定性得到基本保障；它通过项目复盘体现了持续改进思想，使一次课程项目经验能够转化为下一次项目的过程能力。

这个过程的价值不在于形式上完全符合某个标准，而在于能够把标准思想转化为小型项目中真正可执行的做法。对于学生团队来说，最重要的是理解软件过程背后的管理逻辑。需求分析是为了减少误解，设计是为了降低实现复杂度，配置管理是为了保证版本可控，测试是为了发现问题，评审是为了获得反馈，复盘是为了改进能力。只要这些目标能够实现，过程就是有意义的。